

# GuardGrid: A Queriable and Privacy-Preserving Aggregation Scheme for Smart Grid via Function Encryption

Pengfei Yu<sup>1</sup>, Weicong Huang<sup>2</sup>, Ran Zhang<sup>3</sup>, *Student Member, IEEE*,  
Xinyuan Qian<sup>4</sup>, *Graduate Student Member, IEEE*, Hongwei Li<sup>5</sup>, *Fellow, IEEE*,  
and Hanxiao Chen, *Student Member, IEEE*

**Abstract**—Smart grids have revolutionized electricity management by leveraging real-time consumption data, enabling more efficient power control through advanced algorithms. However, this transformation raises significant privacy and security concerns due to the extensive collection of user data. Current solutions face challenges, such as aggregator gateway misbehavior, lack of support for function queries, and the need to balance privacy with efficiency. In this article, we propose FEHH, a novel scheme that ensures both privacy preservation and verifiable aggregation. It allows multiple aggregators to perform inner-product computations on encrypted data while safeguarding the aggregated results from the aggregator. Additionally, it supports verification of aggregated data's correctness using Linear Homomorphic Hash. Building on FEHH, we introduce GuardGrid, a privacy-preserving aggregation scheme for smart grids that inherits FEHH's core features and adds support for essential arithmetic operations necessary for function queries. This allows cloud servers to respond to queries from either the control center or users without compromising data confidentiality. Experimental results show that the encryption overhead of GuardGrid is only 7% of that of the PPDA scheme, and its communication overhead is 123× less. These results demonstrate that GuardGrid significantly reduces computation and communication costs, providing a more sustainable and cost-effective smart grid solution.

**Index Terms**—Aggregation verification, functional encryption (FE), function query, privacy preservation, smart grid.

## I. INTRODUCTION

AS URBANIZATION and information technology advances at an unprecedented pace, smart cities [1], [2], [3] are becoming the ideal paradigm for modern urban planning and development. In this vision, various

intelligent infrastructures are widely deployed to achieve efficient city management and optimize public services. However, the widespread implementation of smart cities also raises significant privacy and security concerns, particularly related to the risks of sensitive data leakage during the collection and sharing processes. Balancing improved city operations with the protection of citizens' privacy has emerged as one of the core challenges in the development of smart cities, driving the need for privacy-preserving technologies.

As a crucial component of smart cities, the smart grid [4], [5], [6] enhances energy distribution and utilization efficiency through advanced sensing, communication, and control technologies. Nevertheless, the extensive deployment of the smart grid introduces privacy risks [7], [8]. Devices, such as smart meters (SMs), continuously collect users' electricity consumption data. While this data is essential for optimizing power resource allocation and enhancing user experience, it can inadvertently expose patterns of behavior and lifestyle choices, leading to significant privacy risks. For instance, Brazil's power company Light S.A. suffered a cyberattack, allowing attackers to analyze power consumption data and infer the daily routines of households [4]. Similarly, a cyberattack on Ukraine's power grid caused widespread power outages. Energias de Portugal (EDP), a multinational energy company, was also targeted, underscoring risks to consumer privacy, as consumption data can be used for targeted promotions [9], possibly against consumers' wishes. Furthermore, malicious actors can exploit low power consumption patterns to determine household absences, heightening risks of theft and fraud [10]. Consequently, safeguarding privacy within the smart grid necessitates robust technical solutions that securely manage users' sensitive data, balancing transparency with effective privacy protection.

In response to these challenges, privacy research has flourished over the past decade [5], [6], [8], [11], [12], [13], [14], aiming to meet the growing demand for privacy protection while also addressing the increasing need for energy data, and thus to strike a practical balance between privacy and utility. Zhang et al. [6] employed the Paillier cryptosystem along with an extended superincreasing sequence for encrypted multidimensional data aggregation. Zhang et al. [13] further enhanced privacy through distributed proxy re-encryption for access control, and zero-knowledge proofs for verification. Wu et al. [5] presented

Received 11 November 2024; revised 5 January 2025; accepted 28 January 2025. Date of publication 7 February 2025; date of current version 23 May 2025. This work was supported by the Science and Technology Project of State Grid Corporation of China (Research on the Theory and Method of Multiparty Encrypted Computation in the Edge Fusion Environment of Power Internet of Things) under Grant 5700-202358592A-3-2-ZN. (Corresponding author: Ran Zhang.)

Pengfei Yu and Weicong Huang are with the State Grid Laboratory of Power Cyber-Security Protection and Monitoring Technology, China Electric Power Research Institute, Nanjing 210003, China (e-mail: yupengfei@geiri.sgcc.com.cn; 864172557@qq.com).

Ran Zhang, Xinyuan Qian, Hongwei Li, and Hanxiao Chen are with the School of Computer Science and Engineering (School of Cyber Security), University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: ranzhang@std.uestc.edu.cn; xinyuanqian@outlook.com; hongweili@uestc.edu.cn; hanxiao.chen@uestc.edu.cn).

Digital Object Identifier 10.1109/IJOT.2025.3539724

a robust and lightweight privacy-preserving data aggregation scheme, PPDA, utilizing a homomorphic threshold proxy re-encryption scheme with re-encryption verifiability (HTV-PRE). However, these existing homomorphic encryption (HE)-based schemes typically impose significant computational burdens due to the expensive operations required by HE. Additionally, they are vulnerable to collusion attacks, where aggregators may collude with users to access other users' private power data.

Furthermore, the ability to make function queries is crucial for users to obtain information from the cloud server (CS) in a smart grid; however, none of the previously mentioned schemes support this functionality under ciphertexts. Liu et al. [15] addressed this issue by enabling the execution of arithmetic circuits to compute complex functions for function queries, all while preserving user privacy. Building on [15], Zhan et al. [12] leveraged the homomorphic properties of the EC-ElGamal encryption algorithm to perform algebraic operations on encrypted data. However, these schemes still rely on cumbersome HE for function queries, which can be computationally intensive, particularly for complex queries that involve multiple multiplications.

**Challenges:** According to the above, designing an efficient and secure data aggregation scheme for the smart grid presents several challenges that must be addressed. First, constructing the scheme that guarantees the correctness of aggregated results while safeguarding their privacy is a significant challenge. On the one hand, the aggregator gate (AG) may fail to aggregate ciphertexts honestly, discarding some user ciphertexts or providing random values to reduce its computational load [5]. On the other hand, the AG might collude with users, compromising the privacy of other users' power consumption data. To mitigate these risks, robust cryptographic techniques are required to verify the correctness of the aggregated results and maintain their confidentiality throughout the aggregation process. However, employing HE introduces complexities, as it necessitates the construction and management of public and private keys for collusion resistance, which can negatively impact overall efficiency. Additionally, traditional functional encryption (FE) schemes tend to inadvertently expose aggregated results to the AG, raising serious concerns about data privacy. Therefore, developing FE methods that effectively protect aggregated results while enabling efficient computations is a critical challenge. Second, designing efficient function query capabilities on encrypted data [15] presents a challenge. Function queries allow the control center (CC) and end users to retrieve specific insights or perform computations on aggregated data, which is essential for smart grid applications. To enable such computations on encrypted data, advanced cryptographic techniques, such as HE, are necessary. However, these techniques often necessitate substantial underlying parameters, which can increase computational complexity and resource consumption [12]. This creates a pressing need for solutions that support secure and efficient function queries without imposing excessive computational burdens. Addressing this challenge requires extensive study to find an optimal balance between functionality and resource efficiency. Third, achieving a balance between privacy and efficiency [16] remains a persistent challenge. Enhancing

privacy typically requires additional cryptographic operations, which slow down system processes. This creates a tradeoff between robust privacy protection and system performance, making it essential to carefully design schemes that maintain both privacy and efficiency without compromising one for the other.

**Contributions:** To tackle these challenges, we propose FEHH, a functional encryption scheme with homomorphic hash, designed to achieve both privacy preservation and verifiable aggregation. This scheme addresses key security challenges by ensuring that aggregated results remain protected while also allowing the correctness of the aggregated data to be verified. Concretely, to protect the aggregated results, FEHH employs a one-time session key  $k$ , which is generated through the Diffie–Hellman (DH) key agreement protocol between two parties, typically the CC and each user. Then each user encrypts their data  $x_i$  by adding their own session key  $k_i$ . The aggregator aggregates the encrypted data and obtains the masked result, which remains concealed by the overall session key  $k$ . This prevents the aggregator from learning the aggregated results during the aggregation process. Furthermore, even if the aggregator behaves maliciously or colludes with some users, it cannot access other users' private data. This is because the aggregator and users do not have access to the individual session keys  $k_i$ , which are required to remove the masking and reveal the original consumption data. In addition to privacy preservation, FEHH incorporates linear homomorphic hash (LHH) to ensure the correctness of the aggregated results. LHH enables the system to verify that the aggregator has performed the aggregation correctly.

Second, building on FEHH, we propose GuardGrid, a robust scheme designed to safeguard both users' data privacy and the correctness of aggregated results in the smart grid ecosystem. GuardGrid inherits the core features of FEHH, including privacy-preserving and verifiable power data aggregation, while introducing additional functionality to enhance computation capabilities. GuardGrid ensures that aggregated results remain hidden from the AG. In this setup, only the CC can access the full aggregated results for further analysis and operations, while the AG remains unaware of the aggregated results during the aggregation process. Furthermore, GuardGrid supports essential arithmetic operations, such as addition (+), subtraction (−), and multiplication (×), which are crucial for function queries. These operations are securely executed on encrypted data, allowing the system to perform necessary calculations while keeping individual users' data confidential. Concretely, GuardGrid introduces a commitment  $cmt$ , computed as  $cmt = g^r$ , where  $r$  is a random nonce. During encryption, this commitment  $cmt$  is incorporated into the ciphertext  $ct$  along with the data  $x$ . The functional key  $sk_f$  is derived from the master secret key and the commitment  $cmt$ , facilitating secure computations on the encrypted data.

Our contributions are summarized as follows.

- 1) We propose FEHH, a verifiable privacy aggregation scheme. It allows external entities to perform aggregation operations on ciphertexts while protecting the aggregated results from the aggregator. Additionally, it

supports verification of the correctness of the aggregated data using LHH.

- 2) We propose GuardGrid, an efficient privacy-preserving data aggregation scheme for smart grid, built upon FEHH. GuardGrid achieves verifiable secure power data aggregation, prevents collusion attacks, and allows the CS to respond to function queries while maintaining data privacy.
- 3) We present a comprehensive theoretical analysis and experimental validation of our scheme. Experiment results show that the encryption overhead of GuardGrid is only 7% of that of the PPDA scheme, and its communication overhead is  $123\times$  lower.

*Organization:* The structure of this article is outlined as follows. Section II provides an overview of related work. The related preliminaries are presented in Section III. Section IV introduces the system model, threat model, and design goals. A comprehensive description of our proposed FEHH and GuardGrid schemes is provided in Sections V and VI, respectively. Section VII provides the correctness and security analysis of our GuardGrid scheme. Section VIII evaluates the performance of the proposed schemes. Section IX concludes the article with a summary of our research.

## II. RELATED WORK

In recent years, user privacy in smart grids [17] has become a significant area of research. Data aggregation schemes have been widely explored, with the majority relying on HE [18] to protect user privacy during electricity consumption data aggregation. HE allows computations on encrypted data, ensuring privacy without compromising functionality. This approach is crucial for smart grids, where safeguarding data privacy is essential, while still enabling efficient energy management and analysis.

Several notable approaches leverage HE for privacy protection in smart grids. For instance, Mohammadali and Haghighi [11] utilized the Paillier cryptosystem for encryption and decryption, along with Nyberg's Accumulator for batch verification and a multidimensional aggregation approach. Similarly, Zhang et al. [6] introduced VPMDA, which employs a modified Paillier cryptosystem and an extended super-increasing sequence for encrypted data aggregation, while also incorporating correctness verification with Boneh–Lynn–Shacham (BLS) signatures. Zhang et al. [13] further enhanced privacy through HE by using digital signatures for authenticity, distributed proxy re-encryption for access control, and zero-knowledge proofs for verification. However, this approach introduces high complexity in managing the distributed proxy re-encryption process. Zhan et al. [12] proposed an EC-ElGamal encryption algorithm with a double trapdoor decryption mechanism, improving encryption efficiency, though the decryption process can be computationally intensive. Additionally, HTV-PRE [5] employs a homomorphic threshold proxy re-encryption scheme with re-encryption verifiability, designed for quantum resistance and suitability for resource-constrained SMs. However, existing schemes that utilize HE often impose significant computational strain due to

the resource-intensive nature of HE operations. These schemes are also vulnerable to collusion attacks, where aggregators may conspire with certain users to illicitly access the private energy consumption data of others. Furthermore, these schemes fail to support functional queries initiated by the CC and users, limiting their practicability in scenarios that require flexible data analysis and monitoring.

Other emerging technologies, such as differential privacy [8] and blockchain [4], are gaining traction in smart grids. Gough et al. [8] presented a differential privacy compliant algorithm that adds noise to SM data to protect privacy, while exploring cost allocation mechanisms using cooperative game theory. However, differential privacy introduces noise that can impact the accuracy of data analysis, requiring careful parameter tuning to maintain a balance between privacy and data utility.

Function query is another key area of focus in privacy-preserving smart grids. Liu et al. [15] presented a fog-computing-based smart grid model that supports functional queries on encrypted usage data, enabling the execution of arithmetic circuits to compute complex functions while preserving user privacy. Zhan et al. [12] leveraged the homomorphic properties of the EC-ElGamal encryption algorithm to perform algebraic operations on encrypted data for functional queries. However, these schemes rely on HE for function queries, which can be computationally intensive, particularly for complex queries involving multiple multiplications.

In summary, the related works propose advanced cryptographic methods to secure data aggregation for smart grids, focusing on privacy, verification, and function queries. However, several challenges remain, including aggregator gateway (AG) misbehavior, the risks of collusion between entities, the lack of support for function queries, and the need to balance privacy with efficiency. Addressing these challenges is essential for achieving a balance between security, efficiency, and practicality in smart grid deployments.

## III. PRELIMINARY

### A. Notations

Table I lists the symbols used in this article and their corresponding meanings.

### B. Functional Encryption

FE [19], [20], [21], [22], [23] is an advanced cryptographic paradigm that enables fine-grained access control to encrypted data. It allows data owners to encrypt their data in a way that enables specific functionalities or computations to be performed on the encrypted data without revealing the underlying content. This is particularly valuable in scenarios where data privacy is critical, yet external parties need to perform certain operations or analyses on the encrypted data.

In this article, we employ single-input FE (SIFE) [24] and multi-input FE (MIFE) [25] to address the challenges of privacy preservation and function queries. Suppose the inner product is defined as follows:

TABLE I  
GENERAL NOTATIONS IN THIS ARTICLE

| Notation                  | Description                                       |
|---------------------------|---------------------------------------------------|
| $\mathbf{x}$              | A vector, represented by bold lowercase letters   |
| $n$                       | The number of users                               |
| $[n]$                     | A set $\{1, 2, \dots, n\}$                        |
| $\lambda$                 | Security parameter                                |
| $k_i$                     | The session key negotiated between each SM and CC |
| $\text{pp}_{\text{MIFE}}$ | The public key of MIFE                            |
| $\text{pp}_{\text{LHH}}$  | The public parameter of LHH                       |
| $sk$                      | The secret key of FEHH                            |
| $h_i$                     | The hash value of user data $x_i$                 |
| $h^*$                     | The linear combination of all hash values $h_i$   |
| $ct_i$                    | The ciphertext of each $SM_i$                     |
| $sk_y$                    | The functional decryption key of FEHH             |
| $C'$                      | The intermediate aggregated result masked by $k$  |
| $C$                       | The actual aggregated result                      |

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i \in [n]} x_i y_i$$

where  $n$  is the length of the vectors  $\mathbf{x}$  and  $\mathbf{y}$ .

The SIFE scheme [26] for computing the inner product functionality  $\mathbf{x} \cdot \mathbf{y}$  is defined as  $\text{SIFE} = \{\text{SIFE.Setup}, \text{SIFE.KeyDer}, \text{SIFE.Enc}, \text{SIFE.Dec}\}$ .

- 1)  $\text{SIFE.Setup}(1^\lambda, 1^n)$ : The algorithm generates a group structure  $(\mathbb{G}, p, g) \leftarrow_{\$} \text{GroupGen}(1^\lambda)$ , and a random vector  $\mathbf{s} = (s_1, \dots, s_n) \leftarrow_{\$} \mathbb{Z}_p^n$ , based on security parameters  $\lambda$  and  $n$ . It defines the public key as  $pk = (g, h_i = g^{s_i})_{i \in [n]}$  and the secret key as  $sk = \mathbf{s}$ . The key pair  $(pk, sk)$  is then output.
- 2)  $\text{SIFE.KeyDer}(sk, \mathbf{y})$ : Using the master secret key  $sk$  and a vector  $\mathbf{y}$ , this algorithm computes and returns the functional secret key  $sk_f = (\mathbf{y}, \mathbf{s})$ .
- 3)  $\text{SIFE.Enc}(pk, \mathbf{x})$ : This algorithm selects a random element  $r \leftarrow_{\$} \mathbb{Z}_p$  and computes  $ct_0 = g^r$ . For each  $i \in [n]$ , it calculates  $ct_i = h_i^r \cdot g^{x_i}$ . The ciphertext is output as  $ct = (ct_0, \{ct_i\}_{i \in [n]})$ .
- 4)  $\text{SIFE.Dec}(pk, ct, sk_f, \mathbf{y})$ : Given the ciphertext  $ct$ , public key  $pk$ , and functional key  $sk_f$  for vector  $\mathbf{y}$ , this algorithm outputs the discrete logarithm base  $g$  of the inner product, i.e.,  $g^{(\mathbf{x} \cdot \mathbf{y})} = \prod_{i \in [n]} ct_i^{y_i} / ct_0^{sk_f}$ .

A scheme  $\Pi$  is IND-secure if any probabilistic polynomial-time (PPT) adversary  $\mathcal{A}$  has a negligible advantage in  $\lambda$ , where the advantage is defined as

$$\text{Adv}_{\Pi}(\lambda, \mathcal{A}) = |\Pr[\mathcal{A}_0^{\Pi}(1^\lambda, \mathcal{A}) = 1] - \Pr[\mathcal{A}_1^{\Pi}(1^\lambda, \mathcal{A}) = 1]|.$$

MIFE [25] enables computations on encrypted data from multiple sources without revealing the underlying data. It extends the concept of SIFE to support multiple inputs, which can be encrypted independently and then combined for decryption to perform a specific function. This multi-input capability makes MIFE particularly well suited for the transmission between the SM and AG where the data come from different and unrelated users. The MIFE scheme is defined as  $\text{MIFE} = \{\text{MIFE.Setup}, \text{MIFE.Enc}, \text{MIFE.KeyGen}, \text{MIFE.Dec}\}$ , as described in Algorithm 1.

#### Algorithm 1: MIFE Scheme

**Input:**  $\lambda :=$  security parameter;  $n :=$  the number of users.

**Output:** Inner-product of  $\mathbf{x}$  and  $\mathbf{y}$ .

```

1 Function MIFE.Setup( $1^\lambda, n$ ):
2    $\mathcal{G} = (\mathbb{G}, p, g) \leftarrow \text{GroupGen}(1^\lambda)$ 
3    $\mathbf{A} \leftarrow_{\mathbb{R}} \mathcal{D}_k, \mathbf{W} \leftarrow_{\mathbb{R}} \mathbb{Z}_p^{m \times (k+1)}$ 
4    $\text{mpk} = (\mathcal{G}, [\mathbf{A}], [\mathbf{W} \mathbf{A}]), \text{msk} = \mathbf{W}$ 
5   return (mpk, msk)

6 Function MIFE.Enc(mpk,  $\mathbf{x} \in \mathbb{Z}_p^m$ ):
7    $r \leftarrow_{\mathbb{R}} \mathbb{Z}_p^k, \mathbf{c} = \begin{pmatrix} -\mathbf{A} \mathbf{r} \\ \mathbf{x} + \mathbf{W} \mathbf{A} \mathbf{r} \end{pmatrix}$ 
8   return  $\text{ct}_{\mathbf{x}} = [\mathbf{c}] \in \mathbb{G}^{k+m+1}$ 

9 Function MIFE.KeyGen(msk,  $\mathbf{y} \in \mathbb{Z}_p^m$ ):
10  return  $\text{sk}_{\mathbf{y}} = \begin{pmatrix} \mathbf{W}^\top \mathbf{y} \\ \mathbf{y} \end{pmatrix} \in \mathbb{Z}_p^{k+m+1}$ 

11 Function MIFE.Dec(mpk,  $\text{ct}_{\mathbf{x}} = [\mathbf{c}], \text{sk}_{\mathbf{y}}$ ):
12   $\mathbf{C} = [\mathbf{c}^\top \text{sk}_{\mathbf{y}}]$ 
13  return  $\log(\mathbf{C})$ 
```

#### C. Linear Homomorphic Hash

LHH [27] allows data to be hashed in a way that operations on the original data are mirrored by corresponding operations on the hash values. This approach preserves the algebraic structure of the data, meaning that operations, such as addition or multiplication applied to the original dataset are reflected in the resulting hash values.

An LHH scheme comprises three polynomial-time algorithms, denoted as  $\text{LHH} = \{\text{LHH.HGen}, \text{LHH.Hash}, \text{LHH.Eval}\}$ . These algorithms are detailed in [28], with the assumption of the hardness of the discrete logarithm problem.

- 1)  $\text{LHH.HGen}(1^\lambda)$ : Given the security parameter  $\lambda$ , it generates the public parameter  $\text{pp}_{\text{LHH}}$ . This parameter includes a description of a cyclic group  $\mathbb{G}$  of prime order  $q$ , a generator  $g \in \mathbb{G}$ . For ease of notation, this public parameter is implicitly the first argument for  $\text{LHH.Hash}$  and  $\text{LHH.Eval}$ .
- 2)  $\text{LHH.Hash}(x)$ : With a message  $x$  as input, this algorithm calculates and outputs the linearly homomorphic hash of  $x : h \leftarrow g^x \in \mathbb{G}$ .
- 3)  $\text{LHH.Eval}(h_1, \dots, h_\ell, \alpha_1, \dots, \alpha_\ell)$ : Given  $\ell$  hashes and  $\ell$  coefficients of linear combination, this algorithm outputs the linear combination of these  $\ell$  hashes:  $h^* \leftarrow \prod_{i \in [\ell]} h_i^{\alpha_i}$ .

#### D. Diffie–Hellman Key Agreement Protocol

In this article, we employ the DH key agreement protocol to aggregate the results securely obtained [29]. The DH Key Agreement Protocol, introduced by Whitfield Diffie and Martin Hellman in 1976 [30], is a foundational method in public-key cryptography [31]. It allows two parties to securely establish a shared secret over an insecure communication



channel without any prior shared secrets. The DH protocol consists of three formal steps.

- 1)  $\text{DH.param}(k)$ : This algorithm generates a group  $\mathbb{G}$  of prime order  $q$ , a generator  $g$ , and a hash function  $H$ .
- 2)  $\text{DH.gen}(\mathbb{G}, g, q, H)$ : This algorithm randomly selects  $x_i \in \mathbb{G}$  as the secret key and defines  $g^{x_i}$  as the corresponding public key.
- 3)  $\text{DH.agree}(x_i, g^{x_j}, H)$ : This algorithm computes and returns the shared seed  $k_{i,j} = H((g^{x_j})^{x_i})$ .

To ensure correctness, if two parties  $P_i$  and  $P_j$  generate key pairs  $(g^{x_i}, x_i)$  and  $(g^{x_j}, x_j)$  using  $\text{DH.gen}$  with identical parameters  $(\mathbb{G}, g, q, H)$ , then  $\text{DH.agree}(x_i, g^{x_j}, H)$  and  $\text{DH.agree}(x_j, g^{x_i}, H)$  should yield the same result. The security of the protocol further requires that an adversary with access to the public keys  $g^{x_i}$  and  $g^{x_j}$ —but neither private key  $x_i$  or  $x_j$ —cannot distinguish the derived shared secret  $k_{i,j}$  from a random value.

#### IV. PROBLEM FORMULATION

##### A. System Model

The system model, illustrated in Fig. 1, consists of five key components[5], [32]: SM, AG, CC, CS, and the trusted third party (TTP). In this model, users within a close geographic area, such as a residential neighborhood, are grouped into clusters, with each group consisting of  $n$  users, each equipped with an SM. The roles and responsibilities of these entities are outlined as follows.

- 1) *SM*: The SM periodically collects electricity consumption data from each user's appliances, such as refrigerators and air conditioners. It individually encrypts and signs the data, then uploads both the encrypted data and hash values to the AG after a brief interval.
- 2) *AG*: The AG is a lightweight edge server with storage and computational capabilities. It receives encrypted data and hash values from the SMs, performs preliminary processing and aggregation, and then forwards the intermediate aggregated results along with the aggregated hash value to the CC.
- 3) *CC*: The CC oversees the entire smart grid, similar to a power system operator. With substantial computing resources, it verifies the correctness of aggregated results, decrypts the data to obtain the actual aggregated results, performs analysis, and adjusts pricing strategies based on the insights gained.
- 4) *CS*: The CS securely stores all data submitted by the CC and handles various function query requests from the CC and end users. Throughout this process, the electricity consumption data remains encrypted, ensuring a high level of security.
- 5) *TTP*: The TTP is typically a trusted government entity and is responsible for initializing the system. It generates and distributes the necessary public and private parameters for all parties in the network. After the initialization phase, the TTP becomes inactive and does not participate in the ongoing data uploads from the users.

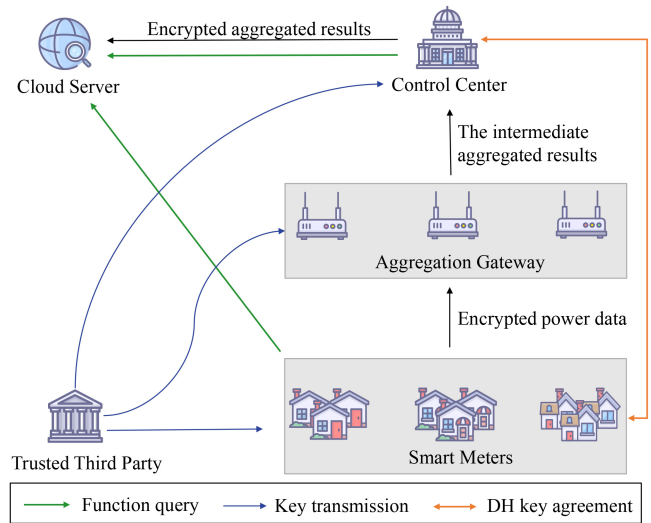


Fig. 1. System model of our GuardGrid scheme.

##### B. Threat Models

In this article, the TTP is considered fully trusted, while both SMs and the CC are regarded as honest-but-curious entities. This means they adhere to the established protocol but may attempt to infer private data. On the other hand, AG is treated as a potentially misbehaving entity. The following potential security threats are considered.

- 1) *Eavesdropping Attack*: During data transmission, an adversary may eavesdrop on the communication channel, intercepting sensitive information and violating user privacy.
- 2) *Semi-Honest Attack*: All honest-but-curious entities are assumed to behave semi-honestly. While they follow the protocol, they may attempt to infer private user information through various indirect means, leading to potential privacy violations.
- 3) *Misbehaving AG*: The AG may fail to aggregate ciphertexts honestly. For example, it could discard some user ciphertexts or provide random values to reduce its computational load. Additionally, the AG may collude with some SMs to compromise data confidentiality.

Similar to [5], attacks, such as data tampering, replay attacks, and false data injection during transmission, are assumed to be mitigated by standard authentication and signature techniques. Therefore, these threats are outside the scope of this work and are not considered further.

##### C. Design Goals

The design of a secure data aggregation scheme for the smart grid ecosystem must adhere to several core security requirements to protect user data privacy and ensure the correctness of aggregated results. The following are the essential goals that the scheme must achieve.

- 1) *Privacy Preservation*: The scheme must ensure the confidentiality of users' electricity consumption data throughout the entire communication process, protecting

it from eavesdropping or interception by potential attackers. Even if an attacker gains access to the aggregated data, they should not be able to deduce the specific consumption data of any individual user.

- 2) *Verifiable Aggregation*: The scheme must verify the correctness of the aggregated result. This ensures that any errors or manipulations by the AG can be promptly identified, maintaining the correctness of the system and preventing fraudulent behavior.
- 3) *Function Query*: The scheme should support secure and efficient function queries, enabling the CC or end users to request specific data or perform computations on the aggregated data without compromising the privacy of individual users.
- 4) *Efficiency*: Given the constrained computational resources of SMs and the frequent intervals at which data is collected, the scheme must prioritize efficiency. This involves minimizing the computational and communication overhead associated with encryption, decryption, and verification tasks, ensuring the system operates smoothly.

## V. FEHH SCHEME

We propose FEHH, a scheme that leverages the combined strengths of FE and LHH, enabling both secure computations and verifiable aggregation. The key advantage of FEHH is that it allows aggregators to perform inner-product computations on encrypted data while simultaneously protecting the aggregated results from the aggregator. Additionally, LHH provides a lightweight verification mechanism for ensuring the correctness of the aggregated data. This approach not only ensures the security of the aggregated results from the aggregator but also guarantees their correctness, effectively addressing the first challenge. The detailed FEHH scheme is shown in Algorithm 2.

As described in Section III-B, the aggregated (inner-product) result,  $agg = \sum_{i \in [n]} x_i y_i$ , is obtained directly after decryption, where  $y_i = 1$  in a smart grid scenario. However, colluding users and the aggregator could potentially access other users' private data by calculating  $agg - x_i$ , which violates privacy-preservation requirements. To address this issue, we employ a one-time session key  $k$ , generated through the DH key agreement protocol between two parties. The DH protocol securely combines the public and private keys of the CC and each user, ensuring that only they can generate the individual session key  $k_i$ . This approach prevents colluding users and aggregators from inferring any private data, thereby maintaining data privacy.

Concretely, each user negotiates with the CC to generate the key  $k_i$ , then encrypts  $x_i + k_i$ , masking their data before sending it to the aggregator. The aggregator then aggregates these encrypted data and obtains the result  $agg = \sum_{i \in [n]} (x_i + k_i)$ , which is masked by the overall session key  $k = \sum_{i \in [n]} k_i$ . This approach prevents the aggregator from accessing the actual aggregated results and protects the privacy of individual users' data, even if the aggregator colludes with certain users, as they cannot uncover private data without knowledge of each user's session key  $k_i$ .

## Algorithm 2: FEHH Algorithm

**Input:**  $\lambda$ := security parameter;  $n$ := the number of users.

**Output:** Inner-product of  $\mathbf{x}$  and  $\mathbf{y}$ .

---

```

1 Function FEHH.Setup( $1^\lambda, 1^n$ ):
2   Execute the DH key agreement protocol to
     generate the one-time session key  $k_i$  between
     each user and the CC;
3   MIFE.Setup( $1^\lambda, n$ )  $\rightarrow$  ( $pp_{MIFE} = mpk, msk$ );
4    $sk = (msk, \{k_i\}_{i \in [n]})$ ;
5   LHH.HGen( $1^\lambda$ )  $\rightarrow$   $pp_{LHH}$ ;
6   return  $sk, pp_{MIFE}, pp_{LHH}$ , and publish
     ( $pp_{MIFE}, pp_{LHH}$ )

7 Function FEHH.Enc( $pp_{MIFE}, sk_i, x_i, i$ ):
8    $x_i = x_i + k_i$ ;
9   MIFE.Enc( $pp_{MIFE}, x_i$ )  $\rightarrow$   $ct_i$ ;
10  LHH.Hash( $x_i$ )  $\rightarrow$   $h_i$ 
11  return ( $ct_i, h_i$ )

12 Function FEHH.KeyDer( $sk, \mathbf{y}$ ):
13  MIFE.KeyGen( $sk, \mathbf{y}$ )  $\rightarrow$   $sk_y$ ;
14  return  $sk_y$ 

15 Function FEHH.Agg( $pp_{MIFE}, \{ct_i\}_{i \in [n]}, sk_y$ ):
16  MIFE.Dec( $pp_{MIFE}, ct_i, sk_y$ )  $\rightarrow$   $C'$ ;
17  LHH.Eval( $h_1, \dots, h_n, \alpha_1, \dots, \alpha_n$ )  $\rightarrow$   $h^*$  where
      $\alpha_i = 1$ ;
18  return ( $C', h^*$ )

19 Function FEHH.Dec( $\{k_i\}_{i \in [n]}, C', h^*$ ):
20  Verify the correctness of the aggregated results:
     LHH.Hash( $C'$ )  $\stackrel{?}{=} h^*$ ;
21   $C = \log(C')$ ;
22   $C = C - \sum_{i \in [n]} k_i$ ;
23  return  $C$ 

```

---

In addition to ensuring privacy preservation, a single FE scheme does not support the verification of aggregated results. This limitation poses a risk, as the aggregator may misbehave by discarding certain user ciphertexts or providing random values to reduce its computational load, thereby undermining the system's subsequent processing and analysis. To address this issue, the CC must verify the aggregation results. To facilitate this verification, the FE scheme incorporates LHH. Specifically, each user computes a hash  $h_i = \text{LHH.Hash}(x_i + k_i)$  and publishes it. This process does not compromise the user's private data  $x_i$  due to the security provided by the session key  $k_i$ . After the aggregation is complete, a homomorphic hash is generated through  $\text{LHH.Eval}(h_1, \dots, h_n, \vec{1})$ , allowing other parties to verify that the aggregation was performed correctly and that no user's data was omitted or manipulated. Finally, the CC computes the hash of intermediate aggregated result  $C'$  and verifies it using the following formula:

$$\text{LHH.Hash}(C') \stackrel{?}{=} \text{LHH.Eval}\left(h_1, \dots, h_n, \underbrace{1, \dots, 1}_{n \text{ items}}\right).$$

If the equation is satisfied, the output is 1; otherwise, it is 0. This mechanism guarantees that the results provided by the aggregator are both reliable and secure.

The correctness of the FEHH scheme means that  $\text{FEHH.Dec}$  can recover the inner product from the output of the aggregator:

for all  $(sk, \text{pp}_{\text{MIFE}}, \text{pp}_{\text{LHH}}) \leftarrow \text{FEHH.Setup}(1^\lambda, 1^n)$ ,  $sk_y \leftarrow \text{FEHH.KeyDer}(sk, y)$ ,  $(ct_i, h_i) \leftarrow \text{FEHH.Enc}(\text{pp}_{\text{MIFE}}, sk_i, x_i, i)$  for all  $i \in [n]$ , and  $(C', h^*) \leftarrow \text{FEHH.Agg}(\text{pp}_{\text{MIFE}}, \{ct_i\}_{i \in [n]}, sk_y)$ , we have that  $\text{FEHH.Dec}(\{k_i\}_{i \in [n]}, C', h^*) = \langle x_1 || \dots || x_n, y \rangle$ , except with negligible probability.

The proof of correctness and security of FEHH is given in Section VII.

## VI. GUARDGRID SCHEME

Building on FEHH, we propose GuardGrid, an advanced data aggregation scheme tailored for the smart grid that enables both privacy protection and efficient function queries. GuardGrid retains the core attributes of FEHH, such as privacy-preserving and verifiable data aggregation. Additionally, GuardGrid enables the CS to manage function queries from either the CC or users without compromising the data's confidentiality.

### A. Detailed Construction of GuardGrid

Consider an aggregation area with  $n$  users, where each user is represented as  $SM_i$  for  $i \in [n]$ . As shown in Fig. 2, GuardGrid consists of four steps.

1) *System Initialization*: In the initial phase, each  $SM_i$  and the CC perform DH key agreement protocol (as described in Section III-D) to generate the one-time session key  $k_i$  for encryption. The TTP sets up the system parameters by running  $\text{FEHH.Setup}(1^\lambda, 1^n)$ , initializing the public parameters  $\text{pp}_{\text{MIFE}}$  for MIFE and  $\text{pp}_{\text{LHH}}$  for LHH. The TTP also generates the master keys of MIFE,  $\{sk_i\}_{i \in [n]}$ , and securely distributes these to each  $SM_i$ . Finally, the TTP publishes the system public keys  $\text{pp}_{\text{MIFE}}$  and  $\text{pp}_{\text{LHH}}$ . In the smart grid scenario, our solution mainly focuses on the data aggregation and function query tasks. Hence, the TTP prepares the function decryption key  $sk_y$  and sends it to the AG, where  $y_i = 1$  for  $i \in [n]$ .

2) *Data Collection*: During data collection, each  $SM_i$  records its power consumption  $x_i$  at each reporting interval. It then encrypts and signs  $x_i$  using  $\text{FEHH.Enc}$  to produce the ciphertext  $ct_i$  and hash  $h_i$ , and transmits  $(ct_i, h_i)$  to the AG. Similar to [12], we assume that the duration of each measurement is relatively short (e.g., 15 min), so the electricity consumption data recorded by each SM is relatively small.

3) *Data Aggregation*: Once all ciphertexts and hash values  $(ct_i, h_i)$  are received, the AG aggregates them using  $\text{FEHH.Agg}$ , and sends the intermediate aggregated result  $C'$  along with aggregated hash  $h^*$  to the CC. As mentioned in Section V, the result is masked by the session key  $k$ , which prevents the AG from accessing the real aggregated results. As a result, the AG cannot infer any specific consumption patterns of individual users.

4) *Decryption and Verification*: Upon receiving the aggregated data  $C'$ , the CC decrypts it using  $\text{FEHH.Dec}$  to obtain

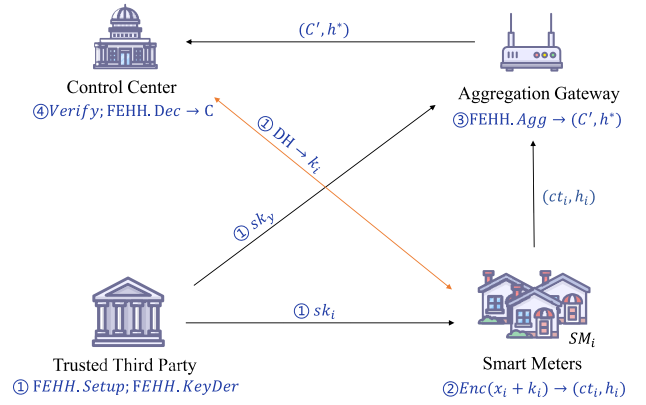


Fig. 2. Detailed construction of our GuardGrid scheme.

the actual aggregated results and simultaneously performs verification to ensure that the aggregation has been correctly executed. If verification fails, the CC discards the aggregation result and logs the AG's incorrect behavior, adjusting its credit score accordingly. If the AG's credit score falls below a set threshold, its service privileges are revoked [5].

### B. Basic Computation for Queries

As described in [12], the CS is equipped to handle function query tasks efficiently, leveraging the encrypted electricity consumption data stored in its database. These function queries can be initiated by the CC or end users, allowing for foundational computations on encrypted electricity data. To achieve this, we extend our FE scheme to support basic computation [26] and propose functional encryption function query (FEFQ). Concretely, the CC encrypts the aggregated results using FEFQ. Encrypt and send the ciphertexts to the CS. Then, CS then performs FEFQ. Decrypt according to function query from the users or CC.

Suppose the basic functionality is defined as follows:

$$f_{\Delta \in \{+, -, *\}}(x, y) = x \Delta y$$

where  $\Delta$  means the basic operation between  $x$  and  $y$ . Our FEFQ scheme is composed of five algorithms and each algorithm is defined as follows.

- 1)  $\text{FEFQ.Setup}(1^\lambda)$ : With the security parameter  $\lambda$  as the input, it outputs the key pair of basic computation  $(\text{mpk}_{\text{BC}}, \text{msk}_{\text{BC}})$ . Concretely, it generates the group samples  $(\mathbb{G}, p, g) \leftarrow_{\$} \text{GroupGen}(1^\lambda)$  and randomly chooses the secret  $s \leftarrow_{\$} \mathbb{Z}_p$ . Subsequently, it sets  $\text{msk}_{\text{BC}} = s$  and  $\text{mpk}_{\text{BC}} = (h = g^s, g)$ .
- 2)  $\text{FEFQ.KeyDerive}(\text{mpk}_{\text{BC}}, \text{msk}_{\text{BC}}, \text{cmt}, \Delta, y)$ : The algorithm takes the master secret key  $\text{msk}_{\text{BC}}$ , the commitment  $\text{cmt}$ , and the function  $y$  as inputs, then outputs the function key  $sk_{f_\Delta}$ . The key generation differs based on the arithmetic operation chosen. The function key  $sk_{f_\Delta}$  is computed as

$$sk_{f_\Delta} = \begin{cases} \text{cmt}^s \cdot g^{-y}, & \text{if } \Delta = + \\ \text{cmt}^s \cdot g^y, & \text{if } \Delta = - \\ (\text{cmt}^s)^y, & \text{if } \Delta = *. \end{cases}$$

- 3)  $\text{FEFQ.Encrypt}(mpk_{BC}, x)$ : Given the public key  $mpk_{BC}$  and  $x$ , the algorithm outputs a commitment  $cmt$ , and the ciphertext  $ct$ . It first randomly selects a nonce  $r \leftarrow \mathbb{Z}_p$ , and computes the commitment  $cmt = g^r$ . The ciphertext is then  $ct = h^r \cdot g^x$ .
- 4)  $\text{FEFQ.Decrypt}(mpk_{BC}, sk_{f_\Delta}, ct, \Delta, y)$ : Based on the public key  $mpk_{BC}$ , the function key  $sk_{f_\Delta}$  for  $f_\Delta$ , the ciphertext  $ct$  and  $y$ , it retrieves the discrete logarithm in basis  $g$  as follows:

$$g^{f_\Delta(x,y)} = \begin{cases} ct/sk, & \text{if } \Delta = + \text{ or } - \\ (ct)^y/sk, & \text{if } \Delta = *. \end{cases}$$

This procedure could be further extended to encompass a more comprehensive scheme, securing not only the raw data but also safeguarding the real query results. Such an extension would align with the principles outlined in Section V, and represents a potential area for future exploration, paving the way for innovative enhancements to our system's capabilities.

## VII. THEORETICAL ANALYSIS

### A. Correctness

Here, we prove the correctness of the aggregation and function query. The correctness of the aggregated result is as follows:

$$\begin{aligned} C &= [\mathbf{c}^\top \mathbf{sk}_y] \\ &= \begin{pmatrix} -\mathbf{Ar} \\ \mathbf{x} + \mathbf{WAr} \end{pmatrix}^\top \begin{pmatrix} \mathbf{W}^\top \mathbf{y} \\ \mathbf{y} \end{pmatrix} \\ &= -\mathbf{ArW}^\top \mathbf{y} + (\mathbf{x} + \mathbf{WAr})^\top \mathbf{y} \\ &= \langle \mathbf{x}, \mathbf{y} \rangle \\ &= \sum_{i \in [n]} \langle x_i, y_i \rangle. \end{aligned}$$

Since the decryption procedure outputs  $\log(C)$ , correctness directly follows.

The correctness of the aggregated results verification is as follows:

$$\begin{aligned} \text{LHH.Hash}(C') &= \text{LHH.Hash}(x_1 + x_2 + \dots + x_n) \\ &= g^{x_1 + x_2 + \dots + x_n} \\ &= g^{x_1} \cdot g^{x_2} \cdot \dots \cdot g^{x_n} \\ &= h_1 \cdot h_2 \cdot \dots \cdot h_n \\ &= \text{LHH.Eval}\left(h_1, \dots, h_n, \underbrace{1, \dots, 1}_{n \text{ items}}\right). \end{aligned}$$

Next, the correctness [26] of function query result is as follows.

For all  $(mpk_{BC}, msk_{BC}) \leftarrow \text{FEFQ.Setup}(1^\lambda)$ , all  $x, y \in \mathbb{Z}_p$ ,  $\Delta \in [+, -, *]$ , for  $sk_{f_\Delta} \leftarrow \text{FEFQ.KeyDerive}(mpk_{BC}, msk_{BC}, cmt, \Delta, y)$ , and  $cmt, ct \leftarrow \text{FEFQ.Encrypt}(mpk_{BC}, x)$ , we have  $\text{FEFQ.Decrypt}(mpk_{BC}, sk_{f_\Delta}, ct, \Delta, y)$  computed as follows:

$$\begin{cases} ct/sk = (h^r g^x) / ((g^r)^s g^{(\mp y)}) = g^{x \pm y}, & \text{if } \Delta = \pm \\ (ct)^y / sk = (h^r g^x)^y / (g^r)^{sy} = g^{xy}, & \text{if } \Delta = *. \end{cases}$$

### B. Security Analysis

After demonstrating the correctness of our scheme, we also prove its security.

First, we define the indistinguishability-based security through the following game with PPT adversary  $\mathcal{A}$ .

- 1) *Initialization*: Run  $(sk, \text{pp}_{\text{MIFE}}, \text{pp}_{\text{LHH}}) \leftarrow \text{FEHH.Setup}(1^\lambda, 1^n)$  and pick a random bit  $b \leftarrow \{0, 1\}$ . Send  $(\text{pp}_{\text{MIFE}}, \text{pp}_{\text{LHH}})$  to  $\mathcal{A}$ .
- 2) *Query*:  $\mathcal{A}$  has access to the following oracles.
  - a)  $\text{OEnc}(i, x_i^0, x_i^1, sk_i)$ : Run  $(ct_i, h_i) \leftarrow \text{FEHH.Enc}(\text{pp}_{\text{MIFE}}, sk_i, x_i^b, i)$ . Return  $ct_i$ .
  - b)  $\text{OKey}(y)$ : Return  $sk_y \leftarrow \text{FEHH.KeyDer}(sk_i, y)$ .
  - c)  $\text{OAgg}(\text{pp}_{\text{MIFE}}, \{ct_i\}_{i \in [n]}, sk_y)$ : Return  $(C', h^*) \leftarrow \text{FEHH.Agg}(\text{pp}_{\text{MIFE}}, \{ct_i\}_{i \in [n]}, sk_y)$ .
- 3) *Finalize*:  $\mathcal{A}$  outputs a bit  $b'$ .

The FEHH scheme is IND-secure if for all  $\mathcal{A}$ , it holds that

$$\text{Adv}_{\mathcal{A}}^{\text{FEHH}}(\lambda) = |\Pr[b = b'] - 1/2| \leq \text{negl}(\lambda).$$

*Theorem 1*: If the MIFE and LHH schemes are secure, the FEHH scheme is IND-CPA secure and computationally verifiable. Formally, for all PPT adversary  $\mathcal{A}$ , there exists a PPT adversary  $\mathcal{B}$  with

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{FEHH}}(\lambda) &\leq \text{Adv}_{\mathcal{B}}^{\text{MIFE}}(\lambda) \\ \text{Adv}_{\mathcal{A}}^{\text{FEHH}}(\lambda) &\leq \text{Adv}_{\mathcal{B}}^{\text{LHH}}(\lambda). \end{aligned}$$

*Proof*: We prove the theorem via a game sequence  $\mathbf{G}_0, \mathbf{G}_1, \mathbf{G}_2$ , where  $\mathbf{G}_0 \approx_s \mathbf{G}_1 \approx_c \mathbf{G}_2$ . Each step is standard, and we provide the games along with a sketch of the proof for the indistinguishability between two adjacent games. In the following, we use boxes to highlight the differences between adjacent games and denote  $\text{Adv}_{\mathcal{A}}^i$  as the advantage of  $\mathcal{A}$  in  $\mathbf{G}_i$  for all  $i \in \{0, 1, 2\}$ .

$\mathbf{G}_0$ : This is the real game with  $b \leftarrow \{0, 1\}$  being the secret bit. Apart from  $(\text{pp}_{\text{MIFE}}, \text{pp}_{\text{LHH}})$ ,  $\mathcal{A}$  gets challenge ciphertexts

$$(ct_i^*, h_i^*) \leftarrow \text{FEHH.Enc}(\text{pp}_{\text{MIFE}}, sk_i, x_i^b, i) \quad \forall i$$

and function decryption key for  $y$

$$sk_y \leftarrow \text{FEHH.KeyGen}(sk_i, y) \quad \forall i$$

Clearly,  $\text{Adv}_{\mathcal{A}}^{\text{FEHH}}(\lambda) = \text{Adv}_{\mathcal{A}}^0(\lambda)$  for all  $\mathcal{A}$ .

$\mathbf{G}_1$ : This game is identical to  $\mathbf{G}_0$  except that we generate all challenge ciphertexts as follows:

$$(ct_i^*, h_i^*) \leftarrow \text{FEHH.Enc}(\text{pp}_{\text{MIFE}}, sk_i, \boxed{k_i}, i), \quad k_i \leftarrow_{\mathbb{R}} \mathbb{Z}_p \quad \forall i.$$

We have  $\mathbf{G}_1 \approx_s \mathbf{G}_0$ ; formally, for all  $\mathcal{A}$ , we have  $\text{Adv}_{\mathcal{A}}^0(\lambda) = \text{Adv}_{\mathcal{A}}^1(\lambda)$ . This follows from the following statistical arguments: for all  $x_i^b \in \mathbb{Z}_p$  and  $y_i$ , it holds that

$$\sum_i (x_i + k_i) \cdot y_i \approx_s \sum_i k_i \cdot y_i.$$

In this game, all challenge ciphertexts do not use the secret random bit  $b$ ; namely, the distribution of  $\mathbf{G}_1$  is independent of  $b$ . Formally, for all  $\mathcal{A}$ , we have  $\text{Adv}_{\mathcal{A}}^1(\lambda) = 0$ . This readily proves that  $\text{Adv}_{\mathcal{A}}^{\text{FEHH}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{MIFE}}(\lambda)$ .



TABLE II  
FUNCTIONALITY COMPARISON

| Functionality      | [6] | [5] | [12] | [4] | GuardGrid |
|--------------------|-----|-----|------|-----|-----------|
| Verify aggregation | ✓   | ✓   | ×    | ×   | ✓         |
| Function query     | ×   | ×   | ✓    | ✓   | ✓         |
| Resist collusion   | ×   | ✓   | ×    | ×   | ✓         |

$\mathbf{G}_2$ : This game is identical to  $\mathbf{G}_1$  except  $\mathcal{A}$  generates  $ct_i \leftarrow_R \mathbb{Z}_p$  for a specific  $i$  and obtains the aggregated result as follows:

$$(C', h^*) \leftarrow \text{FEHH.Agg}(\text{pp}_{\text{MIFE}}, \{ct_i\}_{i \in [n]}, sk_y).$$

To verify, we compute  $\text{LHH.Hash}(C')$ . If  $\text{LHH.Hash}(C') = h^*$ , then  $C'$  is the correct aggregated results. If  $\mathcal{A}$  can forge  $C'$ , it implies  $\mathcal{B}$  can break the LHH scheme. Therefore, the LHH scheme is secure, and the correctness and verifiability of the aggregation results are guaranteed. This readily proves that  $\text{Adv}_{\mathcal{A}}^{\text{FEHH}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{LHH}}(\lambda)$ .

The security of GuardGrid primarily relies on FEHH, and its security is guaranteed as long as the FEHH private key remains undisclosed. Moreover, the security of the function query depends on the underlying work [26], and thus we omit the detailed discussion here. ■

## VIII. EVALUATION

### A. Setup

To evaluate the performance of the proposed GuardGrid, we implemented it in Python, using the GMP library for high-precision calculations essential for cryptographic operations. The experimental environment is configured with an Intel Core Ultra5 125H 1.20-GHz processor and 32 GB of memory. The security parameter is set to  $\lambda = 512$  to ensure a strong level of security. We use the smart grid dataset from [33], an enhanced version of the “Electrical Grid Stability Simulated Dataset” from the UCI Machine Learning Repository. Since our cryptosystem requires integer values, we scaled all data by  $10^3$  to convert the floating-point values to integers. After aggregation, the result was divided by  $10^3$  to restore the correct arithmetic value.

We evaluate GuardGrid on three main aspects: 1) functionality comparison; 2) computation cost, including data encryption on the user side, verifiable aggregation costs, and total computation costs for the SM, AG, and CC; and 3) communication costs between the SM and AG.

### B. Functionality Comparison

We conduct a functionality comparison between GuardGrid and several related schemes, as summarized in Table II. This comparison assesses each scheme’s ability to meet key security requirements, such as resistance to collusion attacks, verification of aggregation correctness, and support for functional queries. As shown in the table, GuardGrid stands out by offering comprehensive security features: it enables verification of the aggregated results, supports functional queries, and resists collusion attacks. GuardGrid provides strong guarantees for data aggregation correctness, allowing

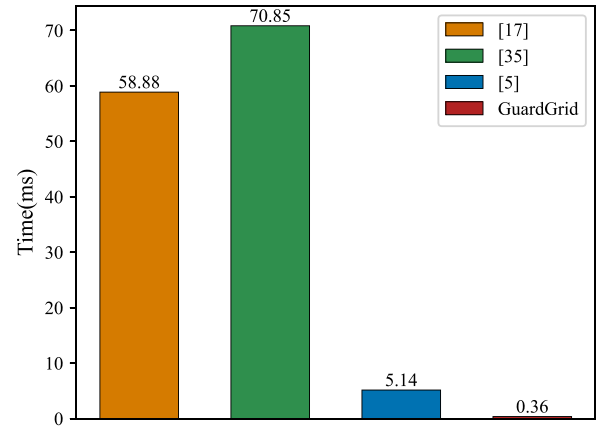


Fig. 3. Data encryption cost of different schemes.

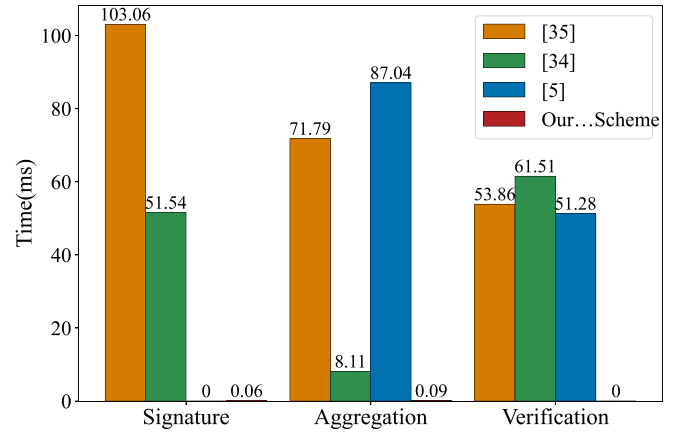


Fig. 4. Verifiable aggregation cost of different schemes.

CC to verify that the aggregator has accurately computed the aggregated ciphertext according to the protocol, ensuring data correctness. Additionally, GuardGrid prevents the aggregator from accessing the aggregated results and protects against potential collusion between the aggregator and users through the use of a one-time secret key.

### C. Computation Cost

1) *Data Encryption Cost*: We evaluate the data encryption cost for each SM, excluding additional computations, such as hash operations to ensure fair comparison [5]. For this analysis, we set the number of SMs in the same residential area to  $n = 1$ . Fig. 3 shows the runtime performance of different aggregation schemes. Since the major computational challenge in a smart grid lies with the SM, efficient encryption is crucial. GuardGrid introduces minimal overhead, achieving significantly lower computational costs compared to existing alternatives. Specifically, the encryption time for GuardGrid is only 0.36 ms, which is 7% of the PPDA scheme’s cost [5], showcasing its superior efficiency.

2) *Verifiable Aggregation Cost*: Once receiving the intermediate aggregated information, the CC decrypts the ciphertext  $C'$  with the secret key  $\sum_{i \in [n]} k_i$ . Our scheme is compared with [5], [34], and [35]. To evaluate the fundamental performance of various verifiable aggregations, the number of SMs,  $n$ , should remain minimal. Here, we choose the

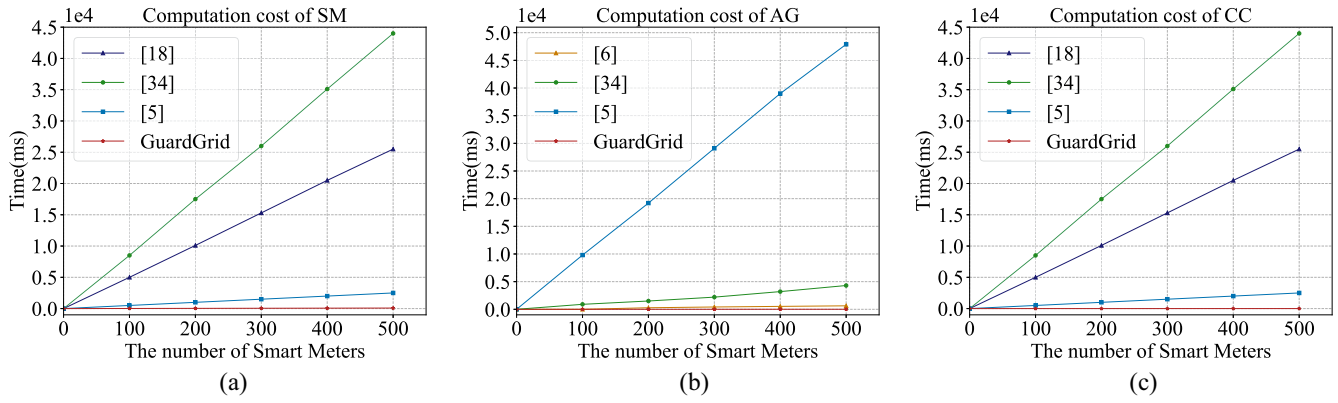


Fig. 5. Computation overhead in (a) SM, (b) AG, and (c) CC side as the number of users increases.

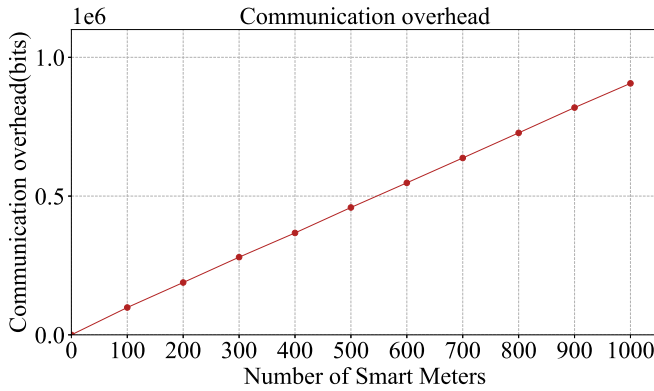


Fig. 6. Communication overhead as the number of users increases.

smallest value,  $n = 1$ . Fig. 4 demonstrates that the verifiable aggregation cost of GuardGrid is significantly lower compared to the other methods.

3) **Total Cost**: The total computation cost includes all encryption and signature operations conducted by SMs, aggregation operations performed by AGs, and decryption and verification tasks executed by CC. Fig. 5 shows that computation cost grows approximately linearly as the number of SMs increases. When the number of SMs reaches 500, our scheme demonstrates impressive efficiency: on the SM side in Fig. 5(a), the computation time is only 3.43% of the PPDA scheme [5]; on the AG side in Fig. 5(b), it accounts for merely 1.33% of the time consumption of the VPMDA scheme [6]; and on the CC side in Fig. 5(c), it requires only 0.07% of the computational resources compared to the PPDA scheme [5]. GuardGrid shows consistently lower computation costs across all components.

#### D. Communication Cost

As SMs are equipped with resource-limited storage and computing devices, the AG often receives encrypted data simultaneously, making the communication costs on each SM  $SM_i$  a crucial factor affecting the performance and feasibility of smart grid deployments. Therefore, we focus on evaluating the communication costs from SMs to the AG, measured in terms of the size of the ciphertext and hash ( $ct_i, h_i$ ). As

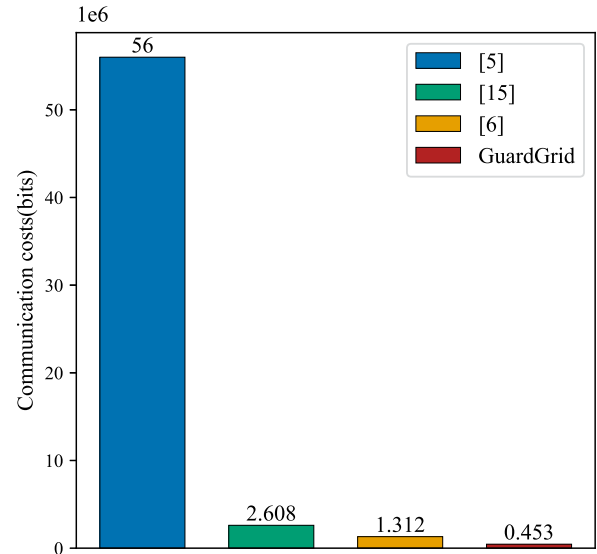


Fig. 7. Communication overhead of different schemes.

illustrated in Fig. 6, the communication overhead increases linearly with the number of users. To compare our scheme with existing ones [6], [5], we set  $n = 500$ , and the detailed comparison of communication costs from  $n$  SMs to the AG is presented in Fig. 7. When the number of SMs reaches 500, the communication costs of our scheme are significantly lower than those of other schemes, representing only 17.37% of Liu et al.'s scheme [15] and 34.53% of Zhang et al.'s scheme [6], and  $123\times$  less than Wu et al. [5].

Since the communication cost becomes irrespective of the number of SMs after aggregation, we omit the experimental results of communication overhead between the AG and CC.

#### IX. CONCLUSION

In this article, we proposed GuardGrid, a privacy-preserving data aggregation framework for smart grids. GuardGrid uses the proposed FEHH scheme to provide verifiably secure power data aggregation, protect against collusion, and enable functional queries while maintaining user privacy. Our experimental results show that GuardGrid substantially enhances efficiency, with an encryption overhead just 7% of that of the

PPDA scheme and a communication overhead reduced by a factor of 123, underscoring its robustness and practicality.

## REFERENCES

- [1] T. Nam and T. A. Pardo, "Conceptualizing smart city with dimensions of technology, people, and institutions," in *Proc. 12th Annu. Int. Digit. Gov. Res. Conf., Digit. Gov. Innov. Chall. Times*, 2011, pp. 282–291.
- [2] M. Farmanbar, K. Parham, Ø. Arild, and C. Rong, "A widespread review of smart grids towards smart cities," *Energies*, vol. 12, no. 23, p. 4484, 2019.
- [3] M. Jafari, A. Kavousi-Fard, T. Chen, and M. Karimi, "A review on digital twin technology in smart grid, transportation system and smart city: Challenges and future," *IEEE Access*, vol. 11, pp. 17471–17484, 2023.
- [4] C. Hu, Z. Liu, R. Li, P. Hu, T. Xiang, and M. Han, "Smart contract assisted privacy-preserving data aggregation and management scheme for smart grid," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2145–2161, Jul./Aug. 2024.
- [5] L. Wu, S. Fu, Y. Luo, H. Yan, H. Shi, and M. Xu, "A robust and lightweight privacy-preserving data aggregation scheme for smart grid," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 270–283, Jan./Feb. 2024.
- [6] X. Zhang, C. Huang, Y. Zhang, and S. Cao, "Enabling verifiable privacy-preserving multi-type data aggregation in smart grids," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 6, pp. 4225–4239, Nov./Dec. 2022.
- [7] M. Z. Gunduz and R. Das, "Cyber-security on smart grid: Threats and potential solutions," *Comput. Netw.*, vol. 169, Mar. 2020, Art. no. 107094.
- [8] M. B. Gough, S. F. Santos, T. AlSkaif, M. S. Javadi, R. Castro, and J. P. S. Catalão, "Preserving privacy of smart meter data in a smart grid environment," *IEEE Trans. Ind. Informat.*, vol. 18, no. 1, pp. 707–718, Jan. 2022.
- [9] P. Kumar, Y. Lin, G. Bai, A. Paverd, J. S. Dong, and A. Martin, "Smart grid metering networks: A survey on security, privacy and open research issues," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2886–2927, 3rd Quart., 2019.
- [10] Y. Chang, J. Li, N. Lu, W. Shi, Z. Su, and W. Meng, "Practical privacy-preserving scheme with fault tolerance for smart grids," *IEEE Internet Things J.*, vol. 11, no. 2, pp. 1990–2005, Jan. 2024.
- [11] A. Mohammadali and M. S. Haghighi, "A privacy-preserving homomorphic scheme with multiple dimensions and fault tolerance for metering data aggregation in smart grid," *IEEE Trans. Smart Grid*, vol. 12, no. 6, pp. 5212–5220, Nov. 2021.
- [12] Y. Zhan, L. Zhou, B. Wang, P. Duan, and B. Zhang, "Efficient function queryable and privacy preserving data aggregation scheme in smart grid," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 3430–3441, Dec. 2022.
- [13] W. Zhang, S. Liu, and Z. Xia, "A distributed privacy-preserving data aggregation scheme for smart grid with fine-grained access control," *J. Inf. Security Appl.*, vol. 66, May 2022, Art. no. 103118.
- [14] M. Akgün, E. U. Soykan, and G. Soykan, "A privacy-preserving scheme for smart grid using trusted execution environment," *IEEE Access*, vol. 11, pp. 9182–9196, 2023.
- [15] J.-N. Liu, J. Weng, A. Yang, Y. Chen, and X. Lin, "Enabling efficient and privacy-preserving aggregation communication and function query for fog computing-based smart grid," *IEEE Trans. Smart Grid*, vol. 11, no. 1, pp. 247–257, Jan. 2020.
- [16] P. H. Mirzaee, M. Shojafar, H. Cruickshank, and R. Tafazolli, "Smart grid security and privacy: From conventional to machine learning issues (threats and countermeasures)," *IEEE Access*, vol. 10, pp. 52922–52954, 2022.
- [17] A. Saleem et al., "FESDA: Fog-enabled secure data aggregation in smart grid IoT network," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6132–6142, Jul. 2020.
- [18] D. Derler, S. Ramacher, and D. Slamanig, "Homomorphic proxy re-authenticators and applications to verifiable multi-user data aggregation," in *Proc. 21st Int. Conf. Financ. Cryptogr. Data Security*, 2017, pp. 124–142.
- [19] D. Boneh, A. Sahai, and B. Waters, "Functional encryption: Definitions and challenges," in *Proc. Theory Cryptogr. Conf.*, 2011, pp. 253–273.
- [20] R. Zhang, H. Li, X. Qian, W. Jiang, and X. Zhang, "An efficient and secure privacy-preserving federated learning via lattice-based functional encryption," in *Proc. IEEE Int. Conf. Commun.*, 2024, pp. 2185–2190.
- [21] X. Qian et al., "Privacy-preserving data evaluation via functional encryption, revisited," in *Proc. IEEE Conf. Comput. Commun.*, 2024, pp. 11–20.
- [22] X. Qian, H. Li, M. Hao, G. Xu, H. Wang, and Y. Fang, "Decentralized multi-client functional encryption for inner product with applications to federated learning," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5781–5796, Nov./Dec. 2024.
- [23] X. Qian, H. Li, M. Hao, S. Yuan, X. Zhang, and S. Guo, "CryptoFE: Practical and privacy-preserving federated learning via functional encryption," in *Proc. IEEE Global Commun. Conf.*, 2022, pp. 2999–3004.
- [24] M. Abdalla, F. Bourse, A. De Caro, and D. Pointcheval, "Simple functional encryption schemes for inner products," in *Proc. IACR Int. Workshop Public Key Cryptogr.*, 2015, pp. 733–751.
- [25] M. Abdalla, D. Catalano, D. Fiore, R. Gay, and B. Ursu, "Multi-input functional encryption for inner products: Function-hiding realizations and constructions without pairings," in *Proc. 38th Annu. Int. Cryptol. Conf.*, 2018, pp. 597–627.
- [26] R. Xu, J. B. Joshi, and C. Li, "CryptoNN: Training neural networks over encrypted data," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 2019, pp. 1199–1209.
- [27] X. Guo et al., "VeriFL: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 1736–1751, 2021.
- [28] M. Bellare, O. Goldreich, and S. Goldwasser, "Incremental cryptography: The case of hashing and signing," in *Proc. 14th Annu. Int. Cryptol. Conf. Adv. Cryptol.*, 1994, pp. 216–233.
- [29] H. Chen et al., "GuardHFL: Privacy guardian for heterogeneous federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 4566–4584.
- [30] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, vol. 22, no. 6, pp. 644–654, Nov. 1976.
- [31] N. Li, "Research on Diffie–Hellman key exchange protocol," in *Proc. 2nd Int. Conf. Comput. Eng. Technol.*, vol. 4, 2010, pp. V4-634–V4-637.
- [32] A. K. Singh and J. Kumar, "A privacy-preserving multidimensional data aggregation scheme with secure query processing for smart grid," *J. Supercomput.*, vol. 79, no. 4, pp. 3750–3770, 2023.
- [33] P. C. Breviglieri (Kaggle, San Francisco, CA, USA). *Smart Grid Stability*. 2021. [Online]. Available: <https://www.kaggle.com/datasets/pcbreviglieri/smart-grid-stability>
- [34] J. Ni, K. Zhang, X. Lin, and X. S. Shen, "Balancing security and efficiency for smart metering against misbehaving collectors," *IEEE Trans. Smart Grid*, vol. 10, no. 2, pp. 1225–1236, Mar. 2019.
- [35] K. Emura, H. Kimura, T. Ohigashi, T. Suzuki, and L. Chen, "Privacy-preserving aggregation of time-series data with public verifiability from simple assumptions and its implementations," *Comput. J.*, vol. 62, no. 4, pp. 614–630, Apr. 2019.

**Pengfei Yu** received the B.S. degree from the University of Science and Technology of China, Hefei, China, in 2007.

He is a Researcher with the State Grid Laboratory of Power Cyber-Security Protection and Monitoring Technology, China Electric Power Research Institute, Nanjing, China. His research interests include power system data security and cryptography.

**Weicong Huang** received the Ph.D. degree in computer science and technology from Nanjing University, Nanjing, China, in 2022.

She is a Researcher with the State Grid Laboratory of Power Cyber-Security Protection and Monitoring Technology, China Electric Power Research Institute, Nanjing. Her research interests include power system data security and cryptography.

**Ran Zhang** (Student Member, IEEE) received the B.S. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2023, where she is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering.

Her research interests include cryptography-based data security and privacy-preserving machine learning.

**Xinyuan Qian** (Graduate Student Member, IEEE) received the B.Eng. degree in security engineering from People's Public Security University of China, Beijing, China, in 2018, and the M.Eng. degree in computer technology from the University of Chinese Academy of Sciences, Beijing, in 2021. He is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China.

He is also a Visiting Researcher with Prof. Fang Yuguang's Lab, City University of Hong Kong, Hong Kong. His research interests include IBE, ABE, FE, applied cryptography, and privacy-preserving machine learning.

**Hongwei Li** (Fellow, IEEE) received the Ph.D. degree from the University of Electronic Science and Technology of China, Chengdu, China, in June 2008.

He is a Professor with the School of Computer Science and Engineering, University of Electronic Science and Technology of China. His research interests include network security and applied cryptography.

Dr. Li won the Best Paper Awards of IEEE ICPADS 2020 and IEEE MASS 2018. He serves as an Associate Editor for IEEE INTERNET OF THINGS JOURNAL. He also serves/served the Technical Symposium Co-Chair for IEEE ICC 2024, ACM TUR-C 2019, and IEEE GLOBECOM 2015. He currently serves as the Vice Chair (conference) of IEEE ComSoc CIS-TC. He is a Distinguished Lecturer of the IEEE Vehicular Technology Society.

**Hanxiao Chen** (Student Member, IEEE) received the Ph.D. degree in cybersecurity from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2024.

She was a visiting Ph.D. student with Nanyang Technological University, Singapore, from December 2021 to December 2023. She is currently a Postdoctoral Fellow with UESTC. She has published many papers in major conferences/journals, including NDSS, USENIX Security, ICML, and IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY. Her research interests include cryptography-based data security and privacy protection.